

Cortical Synaptic Graph Networks

Objective-based plasticity,
bounded graph computation,
and verifiable consolidation

VERSION 0.4.1

16 September 2026

Prediction → Error → Bounded write → Cold recall

Research specification, not an empirical validation.

Local guarantees are stated with their assumptions.

Integrated learning, retention, robustness, and scale remain to be tested.

Includes derivations, a lifecycle protocol, an experiment plan,
and reproducible local mathematical checks.

Contents

Research status and abstract	3
1. Scope, motivation, and relationship to prior work	4
1.1 The research question	4
1.2 Established ingredients and proposed contribution	4
1.3 Four meanings of “works”	5
2. State, notation, and causal clocks	5
2.1 Persistent and transient state	5
2.2 Exact endpoint feasibility and independent decay	5
2.3 Four clocks and one event order	6
3. Context-dependent sparse graph routing	7
3.1 Region and column selection	7
3.2 Gates, null mass, and availability	7
3.3 Temporal service, not merely static connectivity	8
4. Bounded messages and a conditionally stable workspace	8
4.1 Typed messages and weight-independent receiver budgets	8
4.2 Inner recurrent computation	8
5. Define what a plastic write learns	9
5.1 Required target contract	9
5.2 A fixed-feature objective for scalar graph synapses	10
5.3 Independent retention and writing	10
6. Error-correcting updates and their guarantees	11
6.1 Normalized delta rule as a transparent special case	11
6.2 An implementable convex feasible set	11
6.3 Projected graph write	12
6.4 Feasible approximations and finite precision	13
6.5 Local retention constraints	13
6.6 Reachability and conditioning before tuning the learner	13
6.7 Measure downstream write utility	14
7. Interference, capacity, and distinguishable context	14
7.1 Exact interference in the unconstrained block	14
7.2 Conflicts are not all forgetting	15
7.3 Finite capacity	15
8. Delayed feedback, reinforcement learning, and meta-learning signals	15
8.1 Feedback records instead of unspecified correlation traces	15
8.2 Reinforcement learning is a separate guarantee class	15
8.3 Inner partial derivatives and outer meta-gradients	16
8.4 Address stability as an explicit ablation	17
9. Working memory, episodic memory, and provenance	17
9.1 Bounded working memory	17
9.2 Episodic record contract	17
9.3 What provenance tiers do and do not guarantee	18

10. Consolidation: immediate continuity and durable recall	18
10.1 An optional exact transfer	18
10.2 Quantization participates in the forward function	18
10.3 Consolidate behavior with fast state disabled	19
11. Maintenance scheduling, validation, and commit semantics	19
11.1 Pressure and bounded maintenance demand	19
11.2 Synchronous reference protocol	20
11.3 Three validation levels, with explicit cost	21
11.4 Optional formal acceptance	21
12. Budgeted structural adaptation	22
12.1 Edge identity and protected routes	22
12.2 Candidate selection and actual value	22
12.3 Admission and training of new edges	22
13. Complete-lifecycle training and evaluation	22
13.1 Training phases	22
13.2 Small reference profile	23
13.3 Required comparisons	24
13.4 Causal memory interventions and transfer	24
13.5 Metrics and decision gates	25
14. Storage and compute accounting	25
14.1 Count the entire adaptive state	25
14.2 Pending credit creates a workload-dependent lower bound	26
14.3 Scored, executed, and written work differ	26
15. Verification results and limits	26
15.1 Executed checks and evidence classes	26
15.2 Guarantee ledger	27
15.3 Principal unresolved risks	28
16. Conclusion	28
Appendix A. Focused migration from v0.4	28
Appendix B. Historical migration from v0.3	29
Appendix C. Reproducibility and provenance	30
References	31

Research status and abstract

Status: research specification with local mathematical guarantees. The integrated architecture has not been empirically validated. This is a new manuscript, not a report of a trained agent or a claim that the complete system converges. It supersedes v0.4 as the authoritative proposed specification. Earlier releases remain historical evidence, not alternate implementation defaults. The existing implementation is outside its scope.

Cortical Synaptic Graph Networks (CSGN) are proposed sparse recurrent learners with a fixed synapse-slot budget, rapidly writable residual weights, slower consolidated weights, finite working and episodic memory, and optional structural adaptation. The v0.4 line changes the organizing principle from bounded correlation accumulation to **bounded correction of an explicit local error**. Computation, environmental interaction, plastic writes, and consolidation occur on distinct clocks. During each write, its feature snapshot, routing support, target, and feasible set are fixed. This permits exact current-association error identities and projected-gradient descent guarantees for a declared local objective. It does not imply improvement of an arbitrary downstream policy.

The graph uses context-dependent routing with a null route, typed bounded messages, and receiver budgets independent of the writable weights. A sufficient contraction condition is given for the computational workspace during a frozen internal cycle; persistent storage is not forced to contract. Retention and writing are independently controlled. Slow consolidation is evaluated with plastic residuals cleared and transient memory standardized. Instantaneous fast-to-slow transfer, long-term persistence, quantization error, and behavioral improvement are treated as different claims.

The specification includes provenance-aware memory handling, a synchronous reference commit protocol, bounded structural proposals, explicit resource accounting, and a staged evaluation plan. Twenty-four groups of newly executed algebra, feasibility, and gradient checks accompany this revision. They validate local identities and demonstrate counterexamples, not the capabilities of a complete CSGN. The main empirical hypothesis is that learned representations and targeted plastic writes can improve the adaptation–retention trade-off beyond matched fixed-graph, replay-only, and established fast-memory baselines. Success on ideal orthogonal keys is not sufficient evidence for that hypothesis.

Revision 0.4.1. This revision replaces the overly conservative absolute-sum reserve with exact four-endpoint decay-safe feasibility; separates fixed inner variables from retained outer gradient paths; makes target reachability and downstream write utility diagnostic requirements; adds memory transplantation, selective-erasure, negative-control, and rule-transfer experiments; elevates the simple delta-memory comparator; and separates practical empirical validation from formal statistical claims. These changes refine a small complete lifecycle rather than add a large subsystem.

Central design contract. A memory write must identify what it predicts, what evidence supplies its target, which state it changes, and which loss its update can improve. Sleep must demonstrate retained behavior after fast memory is removed. Scale follows those results rather than substituting for them.

1. Scope, motivation, and relationship to prior work

1.1 The research question

Can an agent learn new associations during operation, preserve useful older associations, revise obsolete ones, and consolidate selected knowledge into slower storage while using bounded memory and computation? CSGN tests whether a sparse synaptic graph is a useful substrate for that combination.

The proposal does not require a pretrained language model. It does require a training process that makes perception, context representations, routing, and local teaching signals useful. A randomly initialized self-modifying circuit is not presumed to acquire these capabilities merely by operating for a long time. Biological terminology describes architectural inspiration, not an equivalence between artificial columns, parameters, and biological neurons.

Three hypotheses are distinguished:

H1 — useful writing. Objective-based plastic updates improve held-out adaptation when representations and routing are learned, not supplied as ideal keys.

H2 — useful consolidation. Sleep improves slow-memory recall after residuals and unrelated transient cues are removed, without unacceptable losses on still-valid prior tasks.

H3 — useful structure. Budgeted rewiring improves performance or efficiency over a trained fixed graph after charging for its search, retraining, validation, and data movement.

H3 is optional for the first demonstration. A complete small lifecycle must still include graph computation, writable synapses, slow memory, consolidation, fast-state clearing, and delayed recall. Component ablations isolate causes; they do not replace the integrated experiment.

1.2 Established ingredients and proposed contribution

Differentiable plasticity demonstrates that gradient-based training can make Hebbian plastic connections useful on specific task families [1]. Delta-rule fast-weight memories already provide targeted corrections of key-to-value mappings [2], and gated delta networks combine retention control with such corrections [3]. Test-time-training layers interpret recurrent state updates as optimization of a learning objective [4]. CSGN does not claim invention of these mechanisms.

Derivative-based eligibility and policy-gradient methods provide precedents for associating parameter-specific sensitivities with learning signals [5,6]. Dynamic sparse training supplies a reference for fixed-budget topology adaptation [7]. Importance-based methods motivate retention controls, but they are not guarantees of unlimited lifelong memory [8]. Repeated adaptive validation requires its own statistical discipline [9], and low-precision training results do not automatically establish safe INT4 lifetime learning [10].

The proposed contribution is the **integration and evaluation** of these ideas in a typed sparse graph, with an explicit write objective, phase-specific guarantees, residual-aware consolidation, and honest resource accounting. The local mathematical results below are derived for this specification. Published results cited above are not evidence that the complete CSGN works.

1.3 Four meanings of “works”

A well-defined update, a loss-reducing local update, a useful trained learner, and an efficient lifetime agent are different milestones. Version 0.4.1 specifies the first two under stated assumptions. It proposes experiments for the third and fourth. None of the local propositions establishes general intelligence, complete poisoning resistance, universal non-forgetting, or practical operation at 100 billion stored edges.

2. State, notation, and causal clocks

2.1 Persistent and transient state

There are N columns, each with k_{out} reserved outgoing slots. A slot $e = (i, s)$ has source i , target $j(e)$, message type $\chi(e)$, structural class, and semantic generation identifier. Targets may change only through a validated structural transaction. Dormant slots remain part of the budget:

$$E = Nk_{\text{out}}, \quad k_{\text{out}} = k_{\text{backbone}} + k_{\text{plastic}}. \quad (1)$$

The adaptive state at an external event boundary is

$$\mathcal{Z}_t = (\theta, S, P^a, P^u, h, \mathcal{W}, \mathcal{M}, G, \mathcal{F}, \mathcal{V})_t. \quad (2)$$

Here θ contains shared encoders, message transforms, routing, workspace, and readout parameters; S is slow synaptic storage; P^a and P^u are directly admissible and ephemeral residuals; h is the recurrent workspace; $\mathcal{W}$ is working memory; $\mathcal{M}$ is episodic memory; G is the versioned slot topology; $\mathcal{F}$ contains pending feedback records; and $\mathcal{V}$ contains model versions, quantization scales, timestamps, and controller state.

The effective scalar synapse is

$$w_e = S_e + P_e^a + P_e^u. \quad (3)$$

Residuals are in **effective-weight units**. The v0.3 expression coefficient α is not a separate online state; its scale is absorbed into the residual parameterization. Write rates are controlled separately. This is a redesign of the dynamics, not a claim that all v0.3 trajectories are algebraically equivalent.

Cold slots have no active residual unless a retained residual record is explicitly retrieved. The hot set $\mathcal{H}_t$ includes resident nonzero residuals and pinned pending-credit records. Discarding a nonzero residual is a deliberate memory change, not a transparent cache operation.

2.2 Exact endpoint feasibility and independent decay

For every slot, require

$$\begin{aligned} \max_{a,b \in \{0,1\}} |S_e + aP_e^a + bP_e^u| &\leq w_{\text{max}}, & w_{\text{max}} > 0, \quad p_{\text{max}} > 0. \\ |P_e^a| + |P_e^u| &\leq p_{\text{max}}, \end{aligned} \quad (4)$$

The four tested endpoints are S_e , $S_e + P_e^a$, $S_e + P_e^u$, and $S_e + P_e^a + P_e^u$. This replaces v0.4's stronger condition $|S_e| + |P_e^a| + |P_e^u| \leq w_{\text{max}}$. The residual budget is retained to bound internal cancellation and hot-state use.

Endpoint lemma — exact independent-decay safety. For $\lambda_a, \lambda_u \in [0, 1]$, the decayed value is

$$\begin{aligned} S + \lambda_a P^a + \lambda_u P^u &= (1 - \lambda_a)(1 - \lambda_u)S \\ &\quad + \lambda_a(1 - \lambda_u)(S + P^a) \\ &\quad + (1 - \lambda_a)\lambda_u(S + P^u) \\ &\quad + \lambda_a\lambda_u(S + P^a + P^u). \end{aligned}$$

The coefficients are nonnegative and sum to one, so the four endpoint bounds are sufficient. They are also necessary for safety under *all* independent decays because each endpoint is attained by a decay pair in $\{0, 1\}^2$. Applying additional independent decay to the new residuals simply selects points in the original decay rectangle; feasibility is therefore preserved under repeated decay. The residual absolute-sum budget can only decrease.

Corrective capacity. At $S = 1$, $w_{\max} = 1$, $P^u = 0$, the earlier reserve permits only $P^a = 0$. Equation (4) permits an inward correction such as $P^a = -0.3$ when $p_{\max} \geq 0.3$; all independently decayed values remain between 0.7 and 1. This is extra feasible corrective capacity, not a guarantee of better learned behavior.

The feasible set is convex: the endpoint bounds are linear inequalities, and the residual budget is an absolute-sum inequality. Workspace states satisfy $h_i \in [-1, 1]^d$. Keys, values, contexts, and all arithmetic inputs must be finite. Quantizer ranges, target ranges, and tolerances are declared per experiment. Invalid starting states are rejected or repaired as a separately logged maintenance operation, never silently treated as a valid projection base.

2.3 Four clocks and one event order

Use t for an environmental decision, ℓ for an internal computation round, n for a plastic-write event, and ν for a maintenance transaction. The reference causal order is:

1. Receive o_t and any feedback already available for earlier decisions. Resolve due feedback before the next decision, using the corresponding stored records.
2. Build bounded context from o_t , prior working memory, and permitted retrieved records. Choose the active subgraph and freeze its route support, gates, slow weights, and residuals for this internal cycle.
3. Perform K_{inner} workspace rounds. Produce the prediction or action. Save the write snapshot needed if a target will arrive later.
4. Receive the resulting observation, reward, or label when the environment makes it available. Create a causally valid target record; it cannot modify a decision that preceded that feedback.
5. Apply allowed decay and bounded writes. Maintenance occurs at a declared safe boundary.

The reference online phase freezes θ . Offline/meta-training updates θ across episodes. A deployment that changes encoders or shared transforms during maintenance must rebuild affected contexts and records under the new version; it is an extension, not an invisible implementation detail.

3. Context-dependent sparse graph routing

3.1 Region and column selection

Partition columns into regions. A bounded context can be constructed as

$$c_t = C_{\max} \tanh(C_o x_t + C_w \text{Read}(\mathcal{W}_{t-}) + C_m \text{Retrieve}(\mathcal{M}_t)). \quad (5)$$

Retrieval is capped by an explicit record and byte budget. Disable it during pure synaptic-recall tests. Its key encoder is frozen within an online epoch and versioned across maintenance.

Select regions and then columns using query-key interactions. A local score is

$$a_i^t = \frac{q_\theta(c_t)^\top k_\theta(h_i^{t,0}, d_i)}{\sqrt{d_r}} + b_i^{\text{type}}, \quad (6)$$

where d_i is a fixed or offline-learned column descriptor. Descriptors break otherwise identical initial states. Query and key outputs are normalized or norm-bounded. A shared additive context term would cancel from the ordering and is not used.

For the small reference model, region scores and within-selected-region scores are evaluated exactly. This costs $O(Rd_r)$ plus the scored local columns; it is not mislabeled as sublinear. Region summaries are maintained incrementally when their member states change. At large scale, replacing exact region scoring with a tree or approximate index is a separately measured extension, with search recall, staleness, and index-maintenance costs reported.

Denote active senders by $\mathcal{A}_t$ and updated columns by $\mathcal{U}_t$, including admitted receivers and interface columns. Absolute region, sender, receiver, and edge budgets are primary. The active fraction is a resulting measurement, not an independent free parameter.

3.2 Gates, null mass, and availability

For a slot $e = (i, s)$, calculate its routing logit from the current context, source state, target descriptor/state snapshot, and message type. Region-pair biases must be factorized or stored only for actual regional relations; a dense $R \times R$ table is not a scalable default.

Let $v_e \in [0, 1]$ be structural availability. Exclude $v_e = 0$ from selection. Form a bounded eligible support $\mathcal{S}_i$ using adjusted logits $\ell_e + \log v_e$. Preserve a null route with finite logit $\ell_{i,0}$:

$$\begin{aligned} g_e &= \frac{v_e \exp \ell_e}{\exp \ell_{i,0} + \sum_{u \in \mathcal{S}_i} v_u \exp \ell_u}, \quad e \in \mathcal{S}_i, \\ g_{i,0} &= \frac{\exp \ell_{i,0}}{\exp \ell_{i,0} + \sum_{u \in \mathcal{S}_i} v_u \exp \ell_u}. \end{aligned} \quad (7)$$

Other slot gates are zero. Evaluate the expression with a stable log-sum-exp calculation. Real routes satisfy $\sum_s g_{i,s} \leq 1$. Do not normalize them to total mass one and do not multiply by availability again.

A deterministic receiver-capacity admission mask $b_e \in \{0, 1\}$ may reject routes before computation. Define $a_e = b_e g_e \geq 0$ and freeze it for the internal cycle and its later write snapshot. Rejected mass is unused rather than silently redistributed. The set of scored

candidate edges, executed edges, and subsequently writable edges must be recorded separately.

Zero-availability property. With unchanged selection of incumbents, a new slot with $v_e = 0$ changes neither their gates nor its own transmitted value. This does not make a later hard top- k support switch continuous. Positive-availability admission and pruning are evaluated as whole structural edits.

3.3 Temporal service, not merely static connectivity

A protected backbone supplies structural routes between regions, but a static path is not necessarily executed in useful temporal order. Demand activation queues should promote relevant receivers into future sender sets and record time-respecting path latency. The union of edges across a window is insufficient evidence of temporal reachability.

The reference scheduler has a finite demand queue, maximum admitted work, queue overflow policy, and a maximum internal computation budget. An overflow can defer a request, retain it externally, or reject a plastic write; it must be observable in metrics. If at most K_R distinct regions are activated per step, visiting all R regions requires at least $\lceil R/K_R \rceil$ steps. No constant-latency full-coverage claim is made as storage grows.

4. Bounded messages and a conditionally stable workspace

4.1 Typed messages and weight-independent receiver budgets

For each active sender and required head, use

$$m_{i,q}^{t,\ell} = M_{\max} \tanh(U_q h_i^{t,\ell} + V_q c_t), \quad q \in \{1, \dots, H\}. \quad (8)$$

Thus $\|m_{i,q}\|_2 \leq M = M_{\max} \sqrt{d_m}$. With fixed context, its hidden-state Lipschitz constant is bounded by $L_m = M_{\max} \max_q \|U_q\|_2$. More expressive bounded transforms are extensions requiring their own constant or measured stability regime.

Receiver j uses an envelope independent of the writable values:

$$Z_j = \max \left(1, w_{\max} \sum_{e \rightarrow j} a_e \right), \quad r_j^{t,\ell} = \frac{1}{Z_j} \sum_{e \rightarrow j} a_e w_e m_{i(e),\chi(e)}^{t,\ell}. \quad (9)$$

Proposition 1 — receiver bound. Under Equation (4) and the message bound,

$$\|r_j\|_2 \leq \frac{M \sum_{e \rightarrow j} a_e |w_e|}{Z_j} \leq M. \quad (10)$$

The first inequality is the triangle inequality; the second uses $|w_e| \leq w_{\max}$ and the definition of Z_j . The bound does not depend on in-degree. This envelope can attenuate weak weights more than a realized-weight normalizer; its signal scale is an experimental trade-off, not a claim of universal superiority.

4.2 Inner recurrent computation

For $j \in \mathcal{U}_t$, use a fixed mixing coefficient $0 < \kappa \leq 1$ during the cycle:

$$h_j^{t,\ell+1} = (1 - \kappa)h_j^{t,\ell} + \kappa \tanh(A_h h_j^{t,\ell} + B_r r_j^{t,\ell} + u_{j,t}), \quad (11)$$

where $u_{j,t}$ is a bounded sensory/context/working-memory drive fixed during the cycle. Unupdated columns retain their state. Any declared decay of those states is applied at an event boundary and preserves their box bound. A readout query is derived from c_t , rather than being an unspecified constant; the head predicts from selected final states and permitted memory.

Proposition 2 — box invariance. With initial $h_j \in [-1, 1]^d$, Equation (11) preserves that set. Both endpoints of its coordinatewise convex combination lie in the box. This is a magnitude bound, not a claim about learned behavior.

Proposition 3 — frozen-cycle contraction. Let $a = \|A_h\|_2$ and $b = \|B_r\|_2$. In the block-maximum norm $\|h\|_{\text{bm}} = \max_j \|h_j\|_2$, if

$$a + bL_m < 1, \quad q = 1 - \kappa + \kappa(a + bL_m) < 1, \quad (12)$$

the update restricted to $\mathcal{U}_t$ is contractive for fixed gates, weights, input, context, and identical inactive-column boundary states.

Proof. The coefficient sum at a receiver satisfies $\sum_e a_e |w_e| / Z_j \leq 1$. Consequently its message change is at most $L_m \|\Delta h\|_{\text{bm}}$. Tanh is 1-Lipschitz, so the candidate update changes by at most $(a + bL_m) \|\Delta h\|_{\text{bm}}$. Combining the residual path gives the factor q .

The reference profile enforces this condition by constraining operator norms. For example, $a \leq 0.25$, $bL_m \leq 0.5$, and $\kappa = 0.5$ give $q \leq 0.875$; these are illustrative sufficient values, not tuned recommendations. A less restrictive workspace is allowed only as a separately labeled empirical variant. Sweep gains *within* the sufficient condition before relaxing it; excessive contraction can attenuate useful signals. Record actual operator norms and task performance. An approximate power-iteration estimate is not a certified norm bound unless its estimation error is addressed.

Scope limit. The proposition is not contraction of all persistent memory, the environment, routing switches, or the joint plasticity loop. Frozen inactive coordinates have identity dynamics. Between cycles, inputs and memory can change. Test finite-horizon sensitivity of the augmented state as well as inner-cycle norms; stable diagonal blocks do not imply stable coupled dynamics.

5. Define what a plastic write learns

5.1 Required target contract

Every write record contains a decision identifier, time of prediction, time feedback became available, model/topology/feature versions, feature snapshot, target, provenance class, and finite resource charge. The target must be specified before invoking any learning guarantee.

The **reference association profile** predicts an externally supplied value code from an observable cue and context. The value arrives after the prediction. The predicted vector is a designated receiver message, or a fixed linear decoding of selected messages. Use a squared-error objective on that vector; classification by nearest code is a downstream metric, not itself covered by the squared-error theorem.

The **predictive profile** uses a bounded encoding of a subsequently observed quantity. Its target encoder is frozen within the epoch and does not include the prediction being evaluated. Frozen encoders stabilize coordinates but do not guarantee informative targets or prevent all representation collapse.

A **learned local teacher** for internal receivers is an extension. Its target is trained using task-grounded outer losses and compared against full-gradient references. Decreasing its local loss is not assumed to decrease final action loss. A novelty score, a confidence score, or the network’s own unsupported prediction is not an independently verified target.

5.2 A fixed-feature objective for scalar graph synapses

Capture the actual source messages, gates, receiver budgets, and selected writable slot identities for a decision. They are constants during the write. Group writable incoming edges into a vector z and construct columns

$$[A_n]_{j,e} = \frac{a_e}{Z_j} m_{i(e),\chi(e)}^{\text{snapshot}} \quad \text{in receiver } j\text{'s output coordinates.} \quad (13)$$

Columns are zero outside their receiver block. Subtract the contribution of nonwritable edges, using their post-decay values held fixed for this update, from the target to obtain b_n . A fixed linear output decoder can be absorbed into A_n and b_n . Then

$$\hat{b}_n = A_n z, \quad \ell_n(z) = \frac{1}{2} \|A_n z - b_n\|_2^2, \quad \nabla \ell_n = A_n^\top (A_n z - b_n). \quad (14)$$

This is a convex quadratic in the selected **graph synapses themselves**. It does not add a mandatory external lookup model. The weight-independent normalization in Equation (9) is essential to this linear snapshot representation.

The snapshot is **numerically fixed for the partial derivative with respect to writable synapses**. It is not automatically detached from the outer meta-gradient. During exact short-episode meta-training, retain the continuous dependency of the captured messages, gates, and features on shared parameters; see Section 8.3. During ordinary deployment, those shared parameters are frozen and snapshots need no outer tape. On the next graph computation, hidden states and messages may change, so the theorem below applies to the captured objective, not automatically to the new unrolled network. With delayed feedback it is an objective on the old snapshot evaluated at the current writable values. Version-incompatible records must be reconstructed from admissible experience or rejected, not applied to a reused slot address.

5.3 Independent retention and writing

At write event n , apply declared decay to residual tiers:

$$P^{a,-} = \lambda_n^a P^a, \quad P^{u,-} = \lambda_n^u P^u, \quad 0 \leq \lambda_n^a, \lambda_n^u \leq 1. \quad (15)$$

Persistent admissible associations use $\lambda_n^a = 1$ in the reference retention test. Ephemeral decay may use $\exp(-\Delta t/\tau_u)$ with $\tau_u > 0$. Decay and learning have separate budgets and logs. Equation (4) remains feasible after independent decay.

The write strength $\beta_n \in [0, 1]$ is independent of λ_n . A memory may therefore receive a large event-driven write and retain it without slow EMA attenuation. A lost association

due to intentional decay is not counted as a failure of the subsequent local-descent proposition; all such comparisons explicitly start from the post-decay state.

6. Error-correcting updates and their guarantees

6.1 Normalized delta rule as a transparent special case

Consider an allocated matrix block $W = S + P$, a fixed nonzero key $k \in \mathbb{R}^{d_k}$, and a fixed target $v \in \mathbb{R}^{d_v}$. This block is a useful special case of synaptic memory and a diagnostic benchmark; it need not replace the graph. After optional decay, let $W^- = S + \lambda P$ and $e = v - W^-k$. Propose

$$W^* = W^- + \frac{\beta}{\varepsilon + \|k\|_2^2} ek^\top, \quad 0 \leq \beta \leq 1, \quad \varepsilon \geq 0. \quad (16)$$

Skip a zero key; it provides no write direction. The reference finite-precision implementation uses a positive denominator floor. The exact one-shot case below uses a nonzero key and $\varepsilon = 0$ only to state the ideal identity.

Proposition 4 — current-association correction. With fixed k, v and no active projection or quantization,

$$v - W^*k = \left(1 - \frac{\beta\|k\|_2^2}{\varepsilon + \|k\|_2^2}\right)e. \quad (17)$$

Proof. Substitute Equation (16), multiply the outer product by k , and use $k^\top k = \|k\|_2^2$. The scalar multiplier belongs to $[0, 1]$, so $\frac{1}{2}\|v - W^*k\|^2 \leq \frac{1}{2}\|e\|^2$. For $\beta = 1$ and $\varepsilon = 0$, the residual is zero in real arithmetic.

This is the established delta-rule principle [2,3] expressed with explicit assumptions. It establishes a current-pattern result, not retention of every earlier pattern, global policy improvement, or correct interpretation of the target.

6.2 An implementable convex feasible set

The default graph write changes one provenance tier at a time, with S and the other tier fixed. After decay, let o_e be the other tier's value, p_e^- the chosen residual, and $z_e^- = S_e + o_e + p_e^-$. First verify the unchanged endpoints $|S_e| \leq w_{\max}$ and $|S_e + o_e| \leq w_{\max}$ and $|o_e| \leq p_{\max}$. Define

$$\begin{aligned} R_e &= p_{\max} - |o_e|, \\ L_e &= \max(-R_e, -w_{\max} - S_e, -w_{\max} - S_e - o_e), \\ U_e &= \min(R_e, w_{\max} - S_e, w_{\max} - S_e - o_e). \end{aligned} \quad (18)$$

These are the exact interval bounds for the chosen residual under Equation (4), with the other values fixed. Check $L_e \leq p_e^- \leq U_e$. Allocate nonnegative step allowances d_e before the write, with $\sum_{e \in \mathcal{J}_n} d_e \leq B_n^{\text{write}}$. Then

$$\begin{aligned}
l_e &= \max(S_e + o_e + L_e, z_e^- - d_e), \\
u_e &= \min(S_e + o_e + U_e, z_e^- + d_e), \\
\mathcal{C}_n &= \prod_{e \in \mathcal{I}_n} [l_e, u_e].
\end{aligned} \tag{19}$$

The post-decay state belongs to this box by construction. Nonselected coordinates are fixed. Euclidean projection is exactly componentwise clipping to these intervals. It enforces all four endpoints and the intentional write bound $\|z^+ - z^-\|_1 \leq B_n^{\text{write}}$. This replaces the symmetric residual interval centered at zero used in v0.4; do not implement that old interval and label it v0.4.1.

Storage-only intervals are obtained by omitting $z^- \pm d$. They are used for the reachability diagnostic. Step-limited intervals govern actual writes. Conflating these sets would mistake a small allowed step for a lack of representational capacity. Allowance priorities are bounded allocation heuristics, not usefulness oracles. A zero-width interval is legal; widespread no-progress projection is a diagnostic, not evidence of successful learning.

6.3 Projected graph write

Use the exact sparse quadratic in Equation (14):

$$\begin{aligned}
\eta_n &= \frac{\beta_n}{\|A_n\|_F^2 + \varepsilon_n}, \quad 0 < \beta_n \leq 1, \quad \varepsilon_n > 0, \\
z^+ &= \Pi_{\mathcal{C}_n} \left[z^- - \eta_n A_n^\top (A_n z^- - b_n) \right], \\
P_e^{\text{chosen},+} &= z_e^+ - S_e - o_e.
\end{aligned} \tag{20}$$

Skip the step when $A_n = 0$, the target is unavailable, the record is inadmissible, or the controller chooses $\beta_n = 0$. For nonzero A_n , $L_n = \|A_n\|_2^2 \leq \|A_n\|_F^2$, so $\eta_n \leq 1/L_n$. Frobenius normalization is conservative and cheap. A certified tighter bound can improve step size. Disjoint receiver blocks can use separate step sizes and step allowances when the objective is block-separable. A decoder that mixes receiver outputs can destroy that separability even when incoming edge identities do not overlap; then use the joint design matrix or a justified block method.

Proposition 5 — bounded local descent. Let $\Delta = z^+ - z^-$. Under the fixed-feature, fixed-target, fixed-set assumptions,

$$\ell_n(z^+) \leq \ell_n(z^-) - \left(\frac{1}{\eta_n} - \frac{L_n}{2} \right) \|\Delta\|_2^2. \tag{21}$$

Proof. Projection optimality, using $z^- \in \mathcal{C}_n$, gives $\langle \nabla \ell_n(z^-), \Delta \rangle \leq -\|\Delta\|^2/\eta_n$. The quadratic smoothness inequality gives $\ell_n(z^+) \leq \ell_n(z^-) + \langle \nabla \ell_n(z^-), \Delta \rangle + (L_n/2)\|\Delta\|^2$. Combining them yields Equation (21). Its coefficient is positive for the reference step size. A zero step may indicate an optimum, an inactive constraint face, or exhausted capacity; it does not imply successful task learning.

This theorem applies to the graph's stated local fitting problem. Unfreezing routing, differentiating through a moving encoder, changing the target during the step, or using the original weight-dependent normalizer changes that problem. Such variants require an

actual derivative of their computation and an appropriate step-size or line-search analysis.

6.4 Feasible approximations and finite precision

The reference check suite tests the exact box projection in floating-point arithmetic against a tolerance. An approximate optimizer does not inherit the theorem solely because it is called a projection. If its result satisfies

$$\langle \nabla \ell_n(z^-), \Delta \rangle \leq -\|\Delta\|^2/\eta_n + \epsilon_{\text{proj}}, \quad (22)$$

then Equation (21) gains the additive term ϵ_{proj} . A positive predicted descent smaller than numerical or projection error is not a certified useful improvement. Check the actual captured loss and feasibility; reject nonfinite results, violated budgets, and increases beyond declared tolerance. Hot arithmetic starts in FP32 for correctness experiments; lower precision is a separate ablation.

6.5 Local retention constraints

A write can fit a new pattern while damaging old patterns. Replay can be included in a stacked quadratic with fixed nonnegative weights:

$$\ell_n^{\text{mix}}(z) = \frac{1}{2} \sum_{r \in \mathcal{B}_n} \omega_r \|A_r z - b_r\|_2^2, \quad \omega_r \geq 0. \quad (23)$$

The same proof applies to the stacked weighted design matrix. It guarantees descent of the mixture, not of each constituent loss. Sampling weights define the intended objective. When a different prioritized sampling distribution is used, either apply an appropriate importance correction on its supported records or declare that the optimized distribution has changed.

An optional stronger local preservation set adds constraints such as $\|A_r(z - z^-)\|_2 \leq \epsilon_r$ for protected snapshots. Their intersection with the box is convex but generally not separable. It requires an appropriate solver, projection accuracy accounting, and additional data access. It is not silently implemented by coordinate clipping. No finite snapshot set establishes preservation on every possible old input.

6.6 Reachability and conditioning before tuning the learner

For diagnostic snapshots, define the storage-constrained error floor

$$E_{\text{floor}} = \min_{z \in \mathcal{C}_{\text{storage}}} \|Az - b\|_2^2.$$

Use the same selected tier, fixed other tier, and slot identities as the write, but omit the per-step allowance. Record free-coordinate rank, singular values, an explicitly defined conditioning statistic, active bound constraints, solver status, and achieved residual. A receiver with fewer independent writable message directions than target dimensions cannot fit arbitrary targets. Rank alone is not enough: weight bounds and target scale also matter.

For an independently checked box solution $\hat{z}$, let $y = A\hat{z} - b$ and $c = A^T y$. For the half-squared objective, a computable dual lower bound is

$$D(y) = -\frac{1}{2}\|y\|^2 - b^\top y + \sum_e \min(c_e l_e, c_e u_e).$$

This follows from the Fenchel representation of the squared norm and minimization of a linear function on each interval. The primal value $\frac{1}{2}\|A\hat{z} - b\|^2$ is an upper bound on the optimum when $\hat{z}$ is feasible; $\max(0, D(y))$ is a lower bound in exact arithmetic. Report their gap and numerical tolerance. A large achieved residual without an appropriate lower bound or converged solution does **not** establish an irreducible error floor. A bound-constrained least-squares implementation can use an independently tested numerical solver [13]. Fixed coordinates are removed before solving and reinstated afterward.

Interpretation: a high certified floor suggests deficient features, connectivity, target coding, or storage capacity; a low floor with weak actual progress suggests conditioning, step size, or allowances; good snapshot fit with poor future queries suggests moving-feature or task misalignment. These are diagnoses to test, not automatic causal attributions. Full-snapshot oracle fitting is an offline diagnostic and is never given hidden evaluation labels or used as an undeclared deployment update.

6.7 Measure downstream write utility

Clone the complete pre-write state and all random-generator states. Compare a written and unwritten branch on the same causally valid query stream:

$$U_n = \mathcal{L}_{\text{query}}(\mathcal{Z}_{\text{no write}}) - \mathcal{L}_{\text{query}}(\mathcal{Z}_{\text{write}}).$$

Positive utility indicates better later performance on that stream. Keep this offline probe from altering live state or revealing probe labels to future decisions. Report the distribution of U_n , not only its mean. A locally descending step with negative utility is a concrete warning that the snapshot target or its features are misaligned with later use. For short exact-gradient references, also measure alignment between the proposed update and the negative full query-loss gradient, evaluated from matched state; alignment is local and is not a long-horizon guarantee.

7. Interference, capacity, and distinguishable context

7.1 Exact interference in the unconstrained block

For an old key q , Equation (16) changes the old prediction by

$$(W^* - W^-)q = \frac{\beta e}{\varepsilon + \|k\|_2^2} (k^\top q). \quad (24)$$

Orthogonal keys do not interfere in this ideal unconstrained model. Highly overlapping keys can. The geometry must be learned or engineered; it is not supplied by the word “plasticity.” After projection, Equation (24) need not hold; use the actual change $\Delta W q$ or its norm bound instead.

Restricted existence construction. Let $k_1, \dots, k_m$ be orthonormal, $m \leq d_k$, and let their targets be within the unconstrained representable range. Starting at zero, one full delta write for each key yields $W = \sum_i v_i k_i^\top$. Every $W k_i = v_i$. A later write for key k_j changes only its corresponding association. Exact full transfer $S \leftarrow S + P$, $P \leftarrow 0$ preserves all these answers. This establishes coexistence of learning, retention, selective revision,

and consolidation in a restricted memory, not learned representation discovery from raw observations.

For a counterexample, start with $W = (1, 0)$, use $k = (0.9, \sqrt{1 - 0.9^2})$, and write target -1 with a full delta update. The new target is fitted, but the old prediction on $(1, 0)$ becomes -0.71 . The accompanying tests reproduce both constructions.

7.2 Conflicts are not all forgetting

If the same available context and key must simultaneously satisfy $Wk = v_1$ and $Wk = v_2$ with $v_1 \neq v_2$, no deterministic mapping can satisfy both. Supply a distinguishable task context, entity, time, or observable history; otherwise mark the earlier relation obsolete or use a distributional target. Retention metrics must distinguish still-valid knowledge from intentional correction of outdated knowledge.

The episodic and context systems should store validity intervals, source identity, and supersession relationships where the task supports them. In partially observed tasks, the agent needs enough history to infer context. A benchmark that hides that history cannot attribute every conflict to poor consolidation.

7.3 Finite capacity

With B stored bits there are at most 2^B storage states; more independent arbitrary information cannot be represented exactly without collisions. Rewiring reallocates finite capacity; it does not create unlimited memory. The relevant hypothesis is an improved adaptation–retention frontier at a declared capacity, precision, and retrieval budget.

Useful responses to interference include context-sensitive allocation, rehearsal of representative associations, separation of incompatible codes, higher-rank or additional blocks, and declared forgetting. All consume representation capacity, data, or computation. Learned key diversity should be evaluated jointly with useful generalization: making every example orthogonal is not a general solution when similar examples should share structure.

8. Delayed feedback, reinforcement learning, and meta-learning signals

8.1 Feedback records instead of unspecified correlation traces

For supervised or predictive delayed targets, retain the required fixed-feature record until the target arrives or a declared retention deadline expires. The writable set can include currently inactive sources. A fading timestamp alone does not reconstruct missed updates or identify which earlier decision received the reward.

On a topology or feature-version mismatch, reconstruct the record from retained admissible observations under the new model, or reject it with a logged reason. Reusing an old slot’s scalar record after its target changed is semantic corruption. Pending records are pinned or explicitly spilled; deadlines and overflow cause measured lost-credit events, not silent erasure.

8.2 Reinforcement learning is a separate guarantee class

A scalar reward or advantage is not inherently inadequate. Under appropriate policy-gradient assumptions, a sampled update uses a scalar advantage with parameter-specific sensitivity:

$$\hat{g}_t = \hat{A}_t \nabla_{\vartheta} \log \pi_{\vartheta}(a_t \mid \mathcal{H}_t). \quad (25)$$

The vector derivative, not the dimensionality of the reward, assigns different directions to different parameters [6]. For an adaptive recurrent agent, exact sensitivities include relevant state and learning history. Truncated BPTT, local eligibility approximations, and detached fast-state derivatives must be labeled and compared against small exact references. E-prop provides an architecture-specific precedent, not a plug-in proof for this graph [5].

Bandit feedback does not reveal the full correct action vector; do not quietly use the supervised association target in a reward-only experiment. A value target or auxiliary next-observation target can be defined, but its regression improvement is not a policy-improvement theorem. Arbitrary replay is off-policy. Use a justified off-policy objective, prediction/value fitting, or behavior distillation, and retain behavior probabilities when an importance-based estimator needs them.

8.3 Inner partial derivatives and outer meta-gradients

Let A_{θ} denote features produced by shared parameters. The inner rule computes $\nabla_z \ell = A_{\theta}^{\top}(A_{\theta}z - b)$ while holding those features fixed with respect to z . The resulting update is still a function $z^+(\theta)$. For a query objective,

$$\frac{dJ}{d\theta} = \frac{\partial J}{\partial \theta} + \frac{\partial J}{\partial z^+} \frac{dz^+}{d\theta},$$

with additional recurrent-state paths included by the full unroll. Numerical snapshot immutability does not require deleting this second path. The short-episode reference retains continuous autograd paths through features, selected gates, message construction, rate normalization, and the exact functional projection. External target codes do not require gradients. An intentionally frozen target encoder stays frozen. The reference does not differentiate through future evidence that was unavailable at the decision.

Discrete support selection is fixed within a derivative calculation. Ordinary autodiff gives the exact derivative of the selected smooth branch away from selection boundaries, not an exact gradient through changing top- k membership. State the treatment of routing explicitly: fixed-support references, smooth training relaxations, score estimators, and straight-through surrogates are different experiments. At clip/max/min ties, derivatives can be nonunique. Gradient finite-difference tests must avoid such kinks, cover interior and stable active-face cases separately, and report the nondifferentiable cases rather than claim a false equality.

Use a named `detached_feature_write_path` ablation to remove feature-to-write meta-gradients while retaining direct query paths. Do not accidentally disable all training by running the complete training-time write under a no-gradient context or converting graph-connected values to NumPy. Deployment may use detached snapshots because shared θ is then frozen. Truncated episode gradients and learned local teachers are labeled approximations. Differentiable plasticity and test-time-training supply relevant precedents for optimizing how a network learns [1,4], not a guarantee that this training succeeds.

The outer learner may choose representations, continuous route parameters, allocation, and bounded β_n , but cannot override the local step bound while claiming Proposition 5. A target encoder could become constant, a router could bypass plastic memory, or a readout

could solve training tasks through unintended context. Task-grounded losses, write-utility probes, and memory interventions therefore accompany every claimed learning result. Zero plasticity remains a valid baseline; activity alone is not evidence of useful learning.

8.4 Address stability as an explicit ablation

Compare the default state-dependent address keys against an epoch-stable-address variant. In the latter, the address key depends on stable descriptors and bounded observable cue/context features, excluding hidden activity produced by the currently writable memory. Both use the same context information and address dimension. This isolates feedback in addressing; it does not remove recurrent graph computation.

A further variant freezes write-message features across equivalent cue/context queries while the workspace still processes retrieved information. It is a separate experiment because it changes more than routing. Record drift between write and later read features, support overlap, and the relation to write utility. Stable encoders do not by themselves make state-dependent addresses stable. Retain a variant because of measured benefit, not because either static or dynamic routing sounds inherently preferable.

9. Working memory, episodic memory, and provenance

9.1 Bounded working memory

Working memory has K key-value slots, a no-write action, and explicit retention/eviction metadata. For a selected slot a , use

$$\begin{aligned}\hat{v}_a &= V_{\max} \tanh(E_v(x_t, h_t)), \\ v_a^+ &= (1 - \zeta_a)v_a + \zeta_a \hat{v}_a, \quad 0 \leq \zeta_a \leq 1, \\ \hat{k}_a &= \frac{E_k(x_t, h_t)}{\max(1, \|E_k(x_t, h_t)\|_2)}.\end{aligned}\tag{26}$$

Keys can be replaced or convexly interpolated within the unit ball. Values remain coordinate-bounded when initialized within that bound. Read with a norm-bounded query and a bounded similarity plus declared age bias. A fixed slot count is a storage budget, not a claim that a slot equals one human memory item.

9.2 Episodic record contract

A record identifies its source, times, validity/context, observations or recoverable features, actions, feedback, terminal status, model and key-encoder versions, and direct admissibility classification. It contains enough state or history to reconstruct its declared training/evaluation episode. A short context window is sufficient only when the task or a fading-memory argument justifies that truncation.

The store has a finite byte budget. Sampling and compaction trade off coverage, trust, recency, conflict, and task diversity. Synthetic replay is identified as model-generated and is not promoted to independently verified evidence by repetition. Keep separate losses for still-valid retained knowledge and obsolete assertions. Store next observations and behavior probabilities whenever the chosen objective requires them.

Keys use a stable encoder within each online epoch. Changing its coordinates requires re-embedding from retained inputs or a validated compatibility map. Claiming that an index is approximate does not excuse mismatched feature spaces.

9.3 What provenance tiers do and do not guarantee

P^a and P^u separate **direct write bookkeeping**. The maintenance controller authorizes which class an event may influence. Shared nonlinear hidden states can nevertheless transmit indirect effects of untrusted history into apparently admissible updates. Two buffers are therefore not a proof of clean attribution, selective unlearning, or poisoning immunity.

For stronger promotion control, rebuild a slow candidate from a known checkpoint using independently admissible records and a declared context policy. Discard or reconstruct histories contaminated beyond that policy. Provenance judgments remain fallible; independent evidence, repetition limits, source quotas, and bounded write influence reduce specified risks but do not prove adversarial robustness.

The protected controller, holdout records, version journal, and action supervisor are outside the self-modifying set. Protection concerns access rights and protocol behavior; it does not make their input signals truthful. External actions taken under a bad fast update cannot be undone by restoring weights.

10. Consolidation: immediate continuity and durable recall

10.1 An optional exact transfer

For an admissible transfer fraction $q_e \in [0, 1]$, hold topology, routes, shared parameters, and the other residual tier fixed:

$$S'_e = S_e + q_e P_e^a, \quad P_e^{a'} = (1 - q_e) P_e^a, \quad P_e^{u'} = P_e^u. \quad (27)$$

Proposition 6 — immediate transfer continuity and endpoint feasibility. Equation (27) preserves $S + P^a + P^u$ exactly in real arithmetic. The new slow endpoint $S + qP^a$ lies between the old endpoints S and $S + P^a$. The new slow-plus-ephemeral endpoint $S + qP^a + P^u$ lies between $S + P^u$ and $S + P^a + P^u$. The other two endpoints are unchanged. Each remains bounded by Equation (4), and $(1 - q)|P^a| + |P^u|$ cannot increase. Thus exact transfer remains feasible under the revised endpoint set. This proof does not apply to an additional quantizer without the separate checks in Section 10.2.

This equality concerns the instantaneous effective function under unchanged routing and state. It does not preserve the controller's pressure statistic or all future trajectories. For a single residual with subsequent decay λ and no new writes, transferring qP changes the future effective value relative to no transfer by

$$qP(1 - \lambda^m) \quad (28)$$

after m decay steps. Increased persistence is a purpose of consolidation, not an algebraic defect. Transfer is optional bookkeeping and a possible candidate initialization; it is not by itself evidence that a useful concept has been learned.

10.2 Quantization participates in the forward function

Let Q_b be a declared block quantizer. A compensated transfer is

$$S'_e = Q_b(S_e + q_e P_e^a), \quad P_e^{a'} = P_e^a + S_e - S'_e, \quad P_e^{u'} = P_e^u. \quad (29)$$

Proposition 7 — quantized bookkeeping identity. The sum remains exactly equal in real arithmetic, including when Q_b saturates. Saturation does, however, invalidate

a small-error or bounded-residual promise. Check Equation (4), representable range, and all storage budgets before accepting this candidate. If infeasible, change scale or precision, reduce the transfer when that helps, or reject it. Do not silently clip the compensation.

For a nonsaturating nearest-grid quantizer with spacing Δ_b , each rounding error is at most $\Delta_b/2$. For unbiased stochastic rounding to adjacent grid points the error magnitude can approach Δ_b , with zero conditional mean under the usual in-range construction. Neither fact proves task accuracy. Quantizer-scale changes modify other values in the block and must be included in validation. Low-precision learning research supports testing these mechanisms, not assuming they work at every precision [10].

The compensation must participate in the effective forward weight. An error accumulator used only in a future optimizer does not preserve the current forward function. Any cold residual or exception record is counted as storage. Exact arbitrary real-valued preservation, pure INT4 storage, and removal of every residual cannot all be promised simultaneously.

10.3 Consolidate behavior with fast state disabled

Let $M_{S,G,0}$ denote the candidate with both residual tiers zero. The default sleep fit keeps shared θ fixed and optimizes slow weights on admissible records, followed by limited structural proposals:

$$\begin{aligned} J_{\text{sleep}}(S, G) = & \mathbb{E}_{D_{\text{new}}} \ell(M_{S,G,0}, y) \\ & + \lambda_o \mathbb{E}_{D_{\text{valid old}}} \ell(M_{S,G,0}, y) \\ & + \lambda_d \mathbb{E}_{D_{\text{distill}}} D_{\text{KL}}(p_{\text{ref}} \| p_{S,G,0}) + \lambda_c C(S, G). \end{aligned} \quad (30)$$

All coefficients are nonnegative and declared. The cost term is optional; it is not a surrogate for measured latency. Distillation uses still-valid reference behavior and is not treated as ground truth when that behavior is known to be wrong. The sleep objective is generally nonconvex through the full graph; Proposition 5 is not a convergence theorem for Equation (30).

Cold-recall protocol. Disable residual writes and both residual tiers for the entire evaluation, not just at its first step. Reset or standardize workspace and working memory. Disable episodic answer retrieval when measuring pure synaptic retention. Supply only the context legitimately needed to identify the query. Report a separate retrieval-enabled condition. Otherwise working memory, residual reconstruction, or external lookup could conceal failed consolidation.

A function can be consolidated by fitting a slow candidate without copying its entire transient state. Conversely, copying a scalar can preserve a transient mistake. The claim tested by H2 is useful cold recall, not a high fraction of transferred bytes.

11. Maintenance scheduling, validation, and commit semantics

11.1 Pressure and bounded maintenance demand

Monitor each region's hot population rather than computing a quantile over predominantly zero cold slots. One normalized load is

$$s_e = (|P_e^a| + |P_e^u|)/p_{\text{max}}, \quad S_t = \max_r Q_q\{s_e : e \in \mathcal{H}_t \cap E_r\}. \quad (31)$$

Define an empty region's statistic as zero and monitor the maximum as well as the quantile. Pending-feedback occupancy, rejected writes, trusted probe loss, saturation, and time since the last accepted maintenance also contribute. A weighted pressure score is only an operational policy; it is not a stability theorem.

The controller has entry/exit hysteresis, minimum spacing, maximum maintenance work, a maximum permitted write backlog, and a failure policy. If maintenance repeatedly fails, it can freeze new admissible writes, retain or spill pinned records within budget, discard declared ephemeral state, or request external supervision. It must not enter an unbounded retry loop while losing feedback invisibly. Globally protected data are not densely scanned at every environmental event; monitor costs are included in accounting.

11.2 Synchronous reference protocol

The initial protocol is deliberately synchronous and versioned. Run maintenance at an episode/reset boundary or another externally safe pause. In an embodied setting, a fixed supervisor handles actions during the pause; a model checkpoint cannot rewind the outside world.

MAINTAIN(reference version v)

1. Pause plastic writes; fix the feedback-log cutoff.
2. Snapshot S , residual tiers, G , θ /version, scales, workspace/WM state, pending records, and controller metadata.
3. Build a shadow candidate from admissible records.
Optional transfer must pass its immediate continuity checks.
4. Fit the slow candidate with residuals disabled.
5. Train and evaluate any bounded structural proposal.
6. Check storage, finite arithmetic, graph, and access invariants.
7. Evaluate the fixed candidate on appropriate fresh holdouts, including cold recall and affected live-state conditions.
8. Reconstruct or reset runtime state and reconcile pending records; prepare a complete new-state manifest.
9. If source version is still v and every required check passes, publish the manifest atomically. Otherwise discard the candidate.
10. Resume from the published state; do not apply unresolved old-version feedback to new-generation slots.

Feedback arriving after the snapshot cutoff is buffered with its source version and accounted for before writes resume. Buffer overflow follows the declared external-event policy, never silent loss.

The initial algebraic transfer and subsequent functional learning are different operations. Exact continuity is required for the former. The latter intentionally changes the model and is judged by its task, retention, and live-state constraints. Do not add a compensating residual that silently cancels all useful learned changes merely to force equality with the old model.

A new model cannot inherit arbitrary old hidden states without validation. At reset-boundary commits, reset is explicit. Else reconstruct from a sufficient retained history or use a validated state adapter. A short recent window is sufficient only with a supporting task assumption or fading-memory bound. The first protocol freezes shared θ during deployment; broader parameter changes require correspondingly stronger reconstruction.

An asynchronous extension must log post-snapshot observations and feedback, replay them under the new model when appropriate, and revalidate before publication. Raw weight deltas cannot in general be rebased across changed features or topology. Copy-on-write storage, journals, crash recovery, and rollback latency are part of that extension's measured cost.

11.3 Three validation levels, with explicit cost

The reference experiment separates three levels. **Level 1: every transaction** runs deterministic invariants plus a preregistered empirical screen on development probes, labeled empirical and with no lifetime probability claim. **Level 2: scheduled independent evaluations** use held-out episodes and independently trained seeds to evaluate hypotheses. **Level 3: optional formal acceptance** uses the conditions and sample budgets below. A Level 1 pass is never renamed a Level 3 certificate.

This is a research scheduling choice, not permission to lower safety requirements in a consequential deployment. If an application requires a formal acceptance guarantee, a candidate lacking the required evidence remains inconclusive or rejected. The synchronous simulator reference has no real-world actuation consequence.

For illustration, $J = 4$, $\delta = 0.01$, and a radius of 0.01 in Equation (32) require $n \geq 133,693$ independent episodes. Validation can dominate learning. Record its examples, wall time, and byte reads. A tighter valid statistical method is a separately justified change, not an unrecorded reduction of the radius.

11.4 Optional formal acceptance

Separate replay/training anchors from acceptance data. Let a candidate be fixed before drawing n fresh, conditionally independent evaluation episodes from a declared stratum. Each episode yields a predeclared bounded loss in $[0, 1]$. For paired loss differences $d_i = \ell_{\text{candidate},i} - \ell_{\text{reference},i} \in [-1, 1]$, Hoeffding's inequality gives a valid one-sided radius. For J tested metrics, a conservative two-sided allocation is

$$r_v = \sqrt{\frac{2 \log(2J/\delta_v)}{n}}. \quad (32)$$

Accept retention only when $\bar{d}_{\text{old}} + r_v \leq \epsilon_{\text{ret}}$, and new-task improvement only when $\bar{d}_{\text{new}} + r_v \leq -\eta_{\text{min}}$, in addition to deterministic constraints. A separately justified binomial interval can be used for failure rates. Large radii require more data or an inconclusive decision, not an assumed pass. Report raw errors and hard failures as well as bounded statistical metrics.

A lifetime error-spending schedule can use

$$\delta_v = \frac{6\delta_{\text{life}}}{\pi^2 v^2}, \quad \sum_{v=1}^{\infty} \delta_v = \delta_{\text{life}}. \quad (33)$$

This supports a union bound **only if every constituent test remains valid conditional on prior decisions**. Repeatedly tuning candidates on the same revealed holdout violates that prerequisite unless a suitable reusable-holdout or uniform-guarantee mechanism is supplied [9]. The reference uses fresh acceptance episodes; if only a finite repeatedly used data set is available, report empirical validation without the lifetime statistical claim.

Independent episodes, not correlated time steps from one trajectory, are the sampling units.

These guarantees concern expected loss on specified test distributions. They do not certify every future behavior, unknown task, poisoning strategy, or rare catastrophic event.

12. Budgeted structural adaptation

12.1 Edge identity and protected routes

Each slot has a semantic identity (source, target, head, generation). Reusing a slot increments its generation and resets every associated residual, pending credit reference, importance statistic, age, and routing cache entry. Unresolved feedback is either reconciled before replacement or retained with its old identity and handled explicitly.

Reserve structural backbone slots and enforce declared regional and receiver capacities. A backbone preserves possible routes, not automatically useful signal transmission. Topology checks and temporal service checks therefore remain separate. No source is forced to prune a positive fraction of useful edges.

12.2 Candidate selection and actual value

Use bounded candidate sets from local regions, existing indexed neighbors, two-hop relations, and random exploration. Choose both target and message head. Filter forbidden duplicates, self-relations where disallowed, and capacity violations before sampling. Empty admissible sets mean “no change,” not an undefined softmax. Candidate construction and any gradient proposal cost are charged.

First-order saliency can screen proposals but cannot establish indispensability. At $\ell(w) = \frac{1}{2}(w - 1)^2$, the correct $w = 1$ has $|w\ell'(w)| = 0$ although pruning it incurs loss 1/2. Estimate the loss of actual ablation and validate the full network, including route normalization. Magnitude/gradient methods such as RigL provide important comparison points [7]; CSGN’s proposed search is not assumed superior.

12.3 Admission and training of new edges

A new live slot begins unavailable and with zero semantic state. Its zero-availability behavior follows Equation (7). However, zero availability also gives zero functional gradient through that edge. To learn a useful candidate, enable it with positive availability on the **shadow model**, assign probationary routing capacity, and train it on admissible data before considering publication.

Availability may ramp on a declared schedule, with age advancing independently of successful usage. Reserve probationary support or otherwise account for incumbent displacement. A positive-availability support change is not claimed function-preserving; evaluate the whole candidate. The reference transaction budget caps replaced slots, training steps, data bytes, and affected regions. Rewiring is retained only if it improves H3 beyond matched fixed-topology and standard sparse-training baselines.

13. Complete-lifecycle training and evaluation

13.1 Training phases

The first phase learns bounded interface features, routing, graph computation, and read-out on task-grounded examples. The second enables the projected local writes while shared parameters are trained through manageable episode lengths. The third trains and

evaluates sleep followed by residual clearing and cold recall. Structural adaptation is introduced after that lifecycle is functional, then included in integrated tests and ablations. For a sampled task sequence $\tau_{1:T}$, the intended outer criterion is

$$\min_{\theta, \phi} \mathbb{E}_{\tau_{1:T}} \sum_{t=1}^T \left[\ell_{\text{query},t} + \lambda_{\text{cold}} \ell_{\text{cold},t} + \lambda_{\text{old}} \ell_{\text{valid old},t} + \lambda_{\text{cost}} C_t \right]. \quad (34)$$

ϕ denotes bounded write/allocation and maintenance-policy parameters. The state evolves through the declared compute, write, and maintenance operations. Not every transaction decision is differentiable. Fixed controller choices, outer-loop search, surrogate gradients, and truncated differentiation are labeled by experiment; they are not presented as exact differentiation through an unlimited lifetime. Only locally guaranteed updates retain their analytical step bounds.

Training sequences explicitly include learning, holding, distractors, delayed feedback, maintenance, residual clearing, recall, context changes, and relearning. Hold out cue identities, context combinations, and whole task families as appropriate. Evaluate horizons and delay distributions longer than those used in meta-training.

13.2 Small reference profile

The following fixes the size and scheduling profile of a first experiment, not a validated hyperparameter optimum. Its task-specific encoder and value-code interface must also be selected and recorded:

Quantity	Reference choice
Columns and regions	$N = 512$; 16 regions of 32 columns
Reserved slots	$k_{\text{out}} = 16$; $E = 8,192$
Slot classes	4 protected slots and 12 plastic slots per source
State and messages	$d = 64$, $d_m = 16$, $H = 4$
Active budget	At most 4 selected regions, 64 senders, 4 real executed routes per sender
Inner rounds	$K_{\text{inner}} = 4$ with frozen route support
Workspace regime	Sufficient norm condition in Equation (12)
Online parameters	Shared θ fixed; projected writes to selected residual tier
Initial precision	FP32 slow and hot state; float64 isolated algebra checks
Association targets	Bounded externally supplied value codes, revealed after prediction
Retention test	Admissible residual decay disabled; ephemeral decay separately declared
Maintenance	Synchronous episode-boundary candidate fitting and commit
Rewiring	Fixed in the first baseline; bounded shadow proposals in structural experiments

Choose weight/rate/data budgets before the run and publish them in its manifest. Message targets must lie in a sensible scale relative to the receiver bound; target reachability is measured, not assumed. Hot capacity, pending-feedback lifetime, replay bytes,

and acceptance sample budgets are explicitly swept rather than left unbounded. Pilot defaults in the handoff configs are engineering starting points, not empirical recommendations. Keep the full compute–write–sleep–clear–recall lifecycle. Defer rewiring, online shared-encoder changes, INT4, offloading, asynchronous maintenance, and learned internal teachers until their own prerequisites pass. Reference region scoring is exact and its cost is reported. Subsequent scales can use $N = 4,096$ and then larger systems only after evidence warrants them.

13.3 Required comparisons

Use a static recurrent graph with the same encoders and training budget; a replay-only slow learner; the prior bounded correlation rule as an ablation; normalized delta/block-memory baselines; the v0.4.1 graph writes; and v0.4.1 with/without consolidation and rewiring. Established fast-memory results motivate these comparisons [1–4], not a claim that a particular baseline must lose.

Match available observations, label timing, task identifiers, representation training, total parameter bytes, fast-state bytes, replay exposure, inference rounds, and wall-clock or operation budgets. Give a replay-only baseline the replay compute that sleep consumes. Report both matched-memory and matched-latency comparisons when simultaneous matching is not practical. Test-time interventions that disable writes complement, but do not replace, separately trained baselines.

13.4 Causal memory interventions and transfer

Slow-memory transplantation. Use two agents with identical shared θ , descriptors, readout, topology, initial state, and quantization convention, but different randomly assigned valid association tables. After separate learning/consolidation, transplant only slow synaptic values from donor to recipient. Reset both residuals and all transient state, disable episodic answer retrieval and plastic writes, and evaluate the donor’s legitimate cue/context queries. Recalled associations should track the transplanted memory if the demonstrated knowledge resides there. Do not transfer a decoder or task-ID lookup with the weights and call that synaptic causality. Use fixed topology initially so slot identities are compatible. Independent training of θ would invalidate this simple intervention.

Selective erasure. Select candidate memory groups using training/development data only; compare their erasure with equal-size and magnitude/usage-matched random erasure. Test affected and unaffected query classes with uncertainty. Generic model damage is not selective causal localization. In this experiment erasure is an offline intervention, not a silently deployed structural edit.

Negative controls. Fresh independent labels on every query, with no predictive information in any permitted history, should not yield held-out prediction above the declared chance expectation. Stable randomly assigned cue labels are *not* a nonlearning control: they can legitimately be memorized after feedback. Also randomize support feedback independently of scored query truth to detect leakage, and assert that future targets, task-generator seeds, and hidden map tables never enter observations.

Rule transfer. Include episodes whose feedback identifies a structured context-dependent transformation, then query unseen inputs and revisit the rule after distractors and sleep. The support set must identify the chosen family; do not demand arbitrary linear-map recovery from insufficient independent examples. For example, a signed-permutation linear map can be tested after a full-rank support design. Hold out input

vectors, mappings, and, separately, map families. Shared-representation meta-learning theory motivates specifying such structure [12]; it does not certify this nonlinear graph.

The simple delta-memory comparator is a primary competitor on these tasks. Match information and training budgets; compare matched-memory and matched-latency frontiers when exact simultaneous matching is impractical. Report uncertainty across independently trained seeds, not only resampled queries from one checkpoint [11]. Any transplant or transfer claim requires the integrated trained graph, not the ideal matrix checks in Section 15.

13.5 Metrics and decision gates

Measure current-task error/regret, few-shot adaptation, still-valid old-task loss, cold recall, context-specific revision, retrieval-enabled recall, and generalization to held-out tasks. Add downstream write utility, snapshot floor bounds and solver gaps, active-feature rank/conditioning, address drift, transplantation tracking, selective-erasure specificity, and held-out rule transfer. Also report projection saturation, inactive/no-progress writes, interference versus key similarity, drift across maintenance, roll-back/rejection rates, missed-credit events, routing coverage/latency, and actual bytes and time.

Gate A: local mathematics. Invariants, current-snapshot descent, zero-availability behavior, transfer identities, and numerical tolerances pass their dedicated checks.

Gate B: learned adaptation. Learned features and routing produce reproducible gains beyond static and replay-only models; success with externally orthogonal keys is not sufficient.

Gate C: durable consolidation. Pure slow recall survives residual disabling and transient-state reset on both new and still-valid prior tasks.

Gate D: delayed and changing context. The agent handles delayed targets, distractors, revised facts, and longer horizons without hidden target leakage or impossible identical-context requirements.

Gate E: structural value and cost. Rewiring improves an adaptation–retention or resource frontier after its overhead is charged.

Gate F: robustness and scale. Adversarial, noisy, distribution-shift, precision, and resource-pressure tests do not reveal unacceptable failures, and measured sparse execution is competitive.

A failed gate prompts diagnosis or simplification. A passed gate supports only its tested conditions. Preregister the primary metrics and selection rule; report multiple independent seeds and uncertainty, not only the best run.

14. Storage and compute accounting

14.1 Count the entire adaptive state

Let all b . variables denote **bytes**, not bits. A minimum accounting decomposition is

$$\begin{aligned}
 M_{\text{total}} = & E(b_S + b_{\text{target}} + b_{\text{type}} + b_{\text{meta}}) \\
 & + |\mathcal{H}|(b_{pa} + b_{pu} + b_{\text{hot index}} + b_{\text{hot meta}}) \\
 & + Ndb_h + M_{\text{pending}} + M_{\text{WM}} + M_{\text{episodic}} \\
 & + M_{\theta} + M_{\text{routing}} + M_{\text{training}} + M_{\text{transaction}} + M_{\text{communication}}.
 \end{aligned} \tag{35}$$

Training memory includes gradients, optimizer state, and unrolled activations. Transaction memory includes snapshots, candidate differences, validation, scales, journals, and state migration. Pending records include retained feature matrices or the data and versions needed to reconstruct them. Do not hide a full-precision residual or optimizer value for every cold edge inside an unspecified constant.

Cold INT4 coefficients alone occupy $E/2$ bytes when perfectly packed. At $E = 100$ billion this is 50 GB in decimal units; it says nothing about total residency. Indices, node state, and active histories can dominate. Node count is $N = E/k_{\text{out}}$; target-identifier width must support that actual N . The 86–100B figure remains a possible distributed storage research target, not a first model configuration or a biological equivalence.

14.2 Pending credit creates a workload-dependent lower bound

If m newly distinct edges per step must remain individually eligible for T steps, the union can reach $\min(E, mT)$ edge identities. Multiple event-specific snapshots can require more than one record per edge. A tiny fixed hot fraction cannot simultaneously promise arbitrary delayed feedback, no information loss, and no external spill. Cap delays or arrivals, spill within budget, compress with justified algebra, or declare and measure expired-credit losses.

14.3 Scored, executed, and written work differ

Let $\mathcal{Q}_t$ contain scored edge candidates, $\mathcal{X}_t$ executed edges, $\mathcal{U}_t$ updated nodes, and $\mathcal{J}_n$ the write set. A representative cost ledger is

$$\begin{aligned} C_{\text{decision}} = & C_{\text{route lookup}} + C_{\text{candidate scores}}(|\mathcal{Q}_t|) \\ & + K_{\text{inner}} [C_{\text{messages}} + O(|\mathcal{X}_t|d_m) + C_{\text{workspace}}] \\ & + C_{\text{readout}} + C_{\text{snapshot}} + C_{\text{queues}} + C_{\text{data movement}}. \end{aligned} \quad (36)$$

For a captured sparse quadratic, the gradient costs sparse products by A_n and A_n^T ; box projection is $O(|\mathcal{J}_n|)$. Replay and multiple write iterations multiply these costs. Scoring top- k_g routes can require inspecting all stored outgoing candidates even though only k_g execute. Decoder work, source-head computation, and index refresh must not be omitted.

Sleep is reported separately in replay examples, optimization steps, structural candidates, validation episodes, bytes transferred, peak storage, and elapsed time. With a fixed active fraction, work grows with graph size. Constant active work requires a decreasing fraction and a routing/retrieval mechanism that still finds useful information. Count cache misses, scatter contention, effective bandwidth, and cold-fetch latency; sparse arithmetic counts alone do not establish a systems advantage.

15. Verification results and limits

15.1 Executed checks and evidence classes

The companion verification/check_math_v0_4_1.py was newly executed for this release on CPU with deterministic seed 20260916. Its 24 groups contain 19,438 local cases using NumPy, SciPy, and PyTorch. All passed their programmed tolerances. These are numerical reference and gradient checks, not an integrated CSGN implementation, trained agent, or experimental result for H1–H3.

Check family	Executed cases	Scope
Revised endpoint feasibility, decay and interval equivalence	6,001	Includes saturated slow-weight inward correction
Projected writes under revised endpoints	3,000	Descent inequality, endpoint bounds and write budget
Normalized delta identity	5,000	Fixed keys and targets
Exact and quantized transfer checks	3,001	Feasibility for exact transfer; rejection example for quantization
Receiver and frozen-workspace examples	2,000	Local boundedness and sufficient contraction examples
Reachability references	103	Known floors, fixed coordinates, and primal-dual gaps
Outer-gradient and cross-framework checks	302	Finite differences, detached-path ablation, active-face test, NumPy/PyTorch parity
Other identities and counterexamples	31	Ideal matrix interventions, interference, routing, invalid inputs, statistical-cost arithmetic

Raw results, tolerances, interpreter/library versions, timestamps, and SHA-256 hashes of the actual test sources are retained in `verification/results_v0_4_1.json`. The verification script exits unsuccessfully on a failed assertion. It must not replace failed tests with cached output. Historical v0.4 results are included only in the labeled archive.

The ideal matrix transplantation check does not establish memory transplantation in a trained graph. The gradient check differentiates a small continuous fixed-support write, not discrete router selection or an entire learning lifetime. The diagnostic solver's bounds concern a fixed box-constrained snapshot, not downstream policy capacity. Proofs and their assumptions remain the basis of analytical claims; random trials detect inconsistencies in numerical implementations.

15.2 Guarantee ledger

Statement	Conditions	Not established
Receiver messages remain bounded	Fixed nonnegative route factors, bounded messages/weights, Equation (9)	Informative messages or good routing
Workspace box is invariant	Bounded initial states and convex update	Useful computation
Workspace contracts during a cycle	Frozen conditions and Equation (12)	Global adaptive or environmental stability
Delta write reduces current association error	Fixed key/target, valid rate, no binding projection	Retention of other keys

Statement	Conditions	Not established
Projected graph write reduces captured loss	Fixed quadratic, exact feasible projection, valid step	Improvement of a moving-feature or final policy loss
Transfer preserves the immediate effective sum	Stated bookkeeping and unchanged forward dependencies	Future trajectory equality or durable correctness
Quantized transfer preserves the sum with compensation	Compensation remains in forward state; accepted feasibility	Small residual under saturation or pure cold exactness

15.3 Principal unresolved risks

The largest scientific uncertainty is whether learned local targets and feature geometry make snapshot descent useful for downstream tasks. Other risks include representational collapse, context aliasing, plasticity suppression, excessive interference, stale features, lost delayed credit, and constraints that eliminate capacity before useful learning occurs.

Sleep may fail to fit slow memory, preserve outdated answers, or reject nearly every useful candidate. Routing can leave relevant regions dormant or make computation too shallow. Rewiring can consume validation effort without benefit. Protected controllers can still ingest misleading evidence. Quantization, replay, and sparse data movement can erase any practical advantage. These are empirical questions and specified threat-model questions, not issues solved by adding more bounded scalars.

16. Conclusion

Version 0.4.1 retains CSGN’s sparse graph, fast and slow synaptic state, working and episodic memory, sleep, and structural plasticity, but gives them a clearer division of labor. The graph computes with frozen routing and memory during a bounded cycle. An evidence-backed target defines a local error. A projected update corrects that error within exact endpoint-feasible reserves and write budgets, while continuous outer meta-gradients teach which writes are useful. Sleep fits and evaluates retained behavior with fast state disabled. Structural edits are admitted only after their whole-network effects and costs are tested.

The result is a research specification with meaningful local guarantees and explicit global uncertainties. The v0.4.1 handoff makes those uncertainties testable: determine target reachability before tuning, compare write and no-write continuations, transplant slow memory under controlled state, test rule transfer, and confront the simple-memory baseline. It is not a theorem that CSGN will become a capable lifelong agent. The decisive next evidence is a trained small graph whose learned representations support useful adaptation, whose sleep produces genuine cold recall, and whose performance survives matched baselines, interference, delay, and resource pressure.

Appendix A. Focused migration from v0.4

Area	Required v0.4.1 change
Feasibility, Equations (4), (18), (19)	Four-endpoint bounds, exact asymmetric residual intervals, unchanged separable projection
Transfer, Proposition 6	Replace absolute-sum proof with endpoint-convexity proof
Snapshot and meta-training	Fix the inner variable while retaining outer continuous gradient paths; explicit detach ablation
Local diagnostics	Storage-only reachable floor, solver gap, rank, conditioning, and future write utility
Addressing	Compare state-dependent and stable-address variants without changing context information
Memory evidence	Transplantation, matched erasure, negative controls and unseen-input rule transfer
Baselines	Simple delta memory is a primary competitor; independent training seeds and fair resource accounting
Validation	Empirical per-transaction screen, independent research evaluations, optional formal certificates
Scope	Keep complete small lifecycle; defer complexity without declaring it unnecessary forever

Equation numbers (1)–(36) are retained for implementation traceability; new diagnostics use unnumbered definitions. v0.4 code or constraints are not alternate defaults. Any intentional deviation is recorded against the contract map in the handoff and evaluated as a named variant.

Appendix B. Historical migration from v0.3

v0.3 area	v0.4 disposition
Equations 1–5: graph and activation	Retain fixed slot accounting; specify exact small-scale routing and count search overhead
Equation 6: additive context score	Replace with query–key interaction because the shared additive term cancels
Equations 7–17: messages and gates	Preserve bounded typed messages; include availability in gate selection; use a weight-independent receiver envelope
Equations 18–22: recurrent cell	Separate frozen-cycle computation; provide a sufficient workspace contraction condition
Equations 23–33: correlation/eligibility/fast updates	Replace the default supervised write with a fixed-objective projected update; separate retention and writing; specify delayed records
Equations 34–36: importance/metaplasticity	Treat scores as allocation heuristics; bound write rates; require ablation before structural deletion
Equations 37–43: memory and pressure	Bound memory endpoints; use versioned records, hot-set statistics, explicit overflow and maintenance policies

v0.3 area	v0.4 disposition
Equations 44–49: consolidation and acceptance	Distinguish instantaneous continuity from persistence; include forward quantization compensation, cold recall, state publication, and valid testing
Equations 50–54: structural change	Preserve bounded candidate search; train unavailable new edges on a shadow model before positive live admission
Equation 55: meta-objective	Evaluate complete write–hold–sleep–clear–recall lifecycles, not only wake query loss
Equations 56–59: storage and cost	Count both residual tiers, snapshots, pending histories, node state, scored candidates, and transaction overhead

Several prior audit statements are deliberately narrowed. A scalar reward is not intrinsically inadequate; local correlations are not automatically gradient eligibilities. Two provenance buffers are not complete isolation. Summable test error allowances do not repair an adaptively reused holdout. Strict contraction of every memory coordinate would defeat persistent storage. Instantaneous transfer is not future-trajectory equivalence. Each correction is incorporated into the main specification rather than left as an external caveat.

Appendix C. Reproducibility and provenance

The immediate baseline is the conversation attachment `csgn_paper_v0_4.md`; its SHA-256 is recorded in `provenance/sources.json`. Historical design lineage includes Patwrick/Neuroplastic/Docs/`csgn_paper_v0_3.md`, Git blob `dddf9f7a196bc77270dd10415ba7bb07fb4a4805`, and the objective-based reassessment of 16 September 2026. The package includes the original v0.4 research archive and reassessment archive unchanged and clearly non-authoritative.

The repository metadata and README were read at commit `281746c9ae962634df04ec34b3ba212af11b3cfd` while preparing this handoff. That observed commit is a discovery reference, not a requirement to reset a newer checkout. No repository files were modified or published during package preparation. Codex must inspect the live checkout, existing instructions, uncommitted work, dependencies, and available hardware before implementation.

The package contains this Markdown manuscript, a typeset PDF and LaTeX source, mathematical reference functions, newly executed checks and raw results, implementation contracts, task protocols, experiment configurations, report schemas, a staged Codex prompt, and a revision feedback loop. The reference functions are not a complete CSGN. There is no trained checkpoint, actual learning benchmark, or claimed empirical improvement in this release.

From the extracted package root, use the pinned reference dependency list to reproduce the CPU checks in an isolated environment:

```
python -m pip install -r requirements-reference.txt
python verification/check_math_v0_4_1.py --output verification/rerun.json
python scripts/validate_package.py
```

PyTorch builds and CUDA installation for the later implementation are hardware-dependent. Codex must record its actual environment and use appropriate official installation guidance; old repository GPU scripts are historical, not a tested requirement of this release. GPU-less environments can run all supplied reference checks. Do not silently skip a gradient check and report full verification.

A fresh rerun produces a new result file. Keep the shipped result immutable as release evidence. BLAS and package differences can change last-bit values; declared tolerances and source hashes make those differences inspectable. Package hashes establish byte identity, not correctness of the scientific claims. The archive should contain no credentials, external font files, private data, or undisclosed model outputs.

References

- [1] Miconi, T.; Stanley, K.; Clune, J. (2018). *Differentiable plasticity: training plastic neural networks with backpropagation*. ICML, PMLR 80, 3559–3568. <https://proceedings.mlr.press/v80/miconi18a.html>
- [2] Schlag, I.; Irie, K.; Schmidhuber, J. (2021). *Linear Transformers Are Secretly Fast Weight Programmers*. ICML, PMLR 139, 9355–9366. <https://proceedings.mlr.press/v139/schlag21a.html>
- [3] Yang, S.; Kautz, J.; Hatamizadeh, A. (2025). *Gated Delta Networks: Improving Mamba2 with Delta Rule*. ICLR; arXiv 2412.06464v3. <https://arxiv.org/abs/2412.06464>
- [4] Sun, Y. et al. (2025). *Learning to (Learn at Test Time): RNNs with Expressive Hidden States*. ICML, PMLR 267, 57503–57522. <https://proceedings.mlr.press/v267/sun25h.html>
- [5] Bellec, G.; Scherr, F.; Subramoney, A.; Hajek, E.; Salaj, D.; Legenstein, R.; Maass, W. (2020). *A solution to the learning dilemma for recurrent networks of spiking neurons*. Nature Communications 11, 3625. <https://doi.org/10.1038/s41467-020-17236-y>
- [6] Sutton, R. S.; McAllester, D. A.; Singh, S. P.; Mansour, Y. (1999). *Policy Gradient Methods for Reinforcement Learning with Function Approximation*. Advances in Neural Information Processing Systems 12. <https://papers.nips.cc/paper/1999/hash/464d828b85b0bed98e80ade0a5c43b0f-Abstract.html>
- [7] Evci, U.; Gale, T.; Menick, J.; Castro, P. S.; Elsen, E. (2020). *Rigging the Lottery: Making All Tickets Winners*. ICML, PMLR 119, 2943–2952. <https://proceedings.mlr.press/v119/evci20a.html>
- [8] Zenke, F.; Poole, B.; Ganguli, S. (2017). *Continual Learning Through Synaptic Intelligence*. ICML, PMLR 70, 3987–3995. <https://proceedings.mlr.press/v70/zenke17a.html>
- [9] Dwork, C.; Feldman, V.; Hardt, M.; Pitassi, T.; Reingold, O.; Roth, A. (2015). *The reusable holdout: Preserving validity in adaptive data analysis*. Science 349, 636–638. Author research record: <https://research.ibm.com/publications/the-reusable-holdout-preserving-validity-in-adaptive-data-analysis>
- [10] Gupta, S.; Agrawal, A.; Gopalakrishnan, K.; Narayanan, P. (2015). *Deep Learning with Limited Numerical Precision*. ICML, PMLR 37, 1737–1746. <https://proceedings.mlr.press/v37/gupta15.html>
- [11] Agarwal, R.; Schwarzer, M.; Castro, P. S.; Courville, A.; Bellemare, M. G. (2021). *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. NeurIPS 34. <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>
- [12] Tripuraneni, N.; Jin, C.; Jordan, M. (2021). *Provable Meta-Learning of Linear Representations*. ICML, PMLR 139, 10434–10443. <https://proceedings.mlr.press/v139/tripuraneni21a.html>
- [13] SciPy developers. *scipy.optimize.lsqr_linear — bounded linear least squares*. Official documentation, accessed 16 September 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.lsqr_linear.html